

La Programación Orientada a Objetos es un cambio de paradigma fundamental y merece un material de estudio detallado. Aquí tienes una propuesta extensa que busca explicar los "por qué" y los "cómo" de la POO en C#.

Material de Estudio: Programación Orientada a Objetos (POO) con C#

Profesor: Jorge Izaguirre **Materia:** Programacion .NET **Curso:** 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: Fundamentos - Pensando en Objetos

(1.1) ¿Por Qué la Programación Orientada a Objetos (POO)?

Hasta ahora, hemos trabajado principalmente con un enfoque *procedural*: definimos datos (variables) y luego escribimos procedimientos (métodos o funciones) que operan sobre esos datos. Esto funciona bien para programas pequeños, pero a medida que los sistemas crecen en complejidad, surgen problemas:

- **Código Desorganizado:** Datos y las funciones que los manipulan pueden estar dispersos por todo el programa, dificultando entender las relaciones y el flujo.
- **Baja Reutilización:** Es común terminar copiando y pegando bloques de código similares en diferentes lugares.
- **Difícil Mantenimiento:** Un cambio en la estructura de los datos puede requerir modificar muchas funciones diferentes.
- **No Modela Bien el Mundo Real:** El mundo real está lleno de "objetos" (personas, autos, facturas, productos) que tienen características (datos) y pueden realizar acciones (comportamientos). El enfoque procedural no refleja esta realidad de forma natural.

La **POO** propone un cambio de paradigma: organizar el software alrededor de **objetos**, que son entidades que encapsulan tanto los **datos (estado)** como los **comportamientos (métodos)** que operan sobre esos datos.

- *¿Por qué este enfoque es mejor para sistemas complejos?*
 - **Modularidad:** Los objetos son unidades autocontenidas, más fáciles de desarrollar, probar y mantener de forma independiente.
 - **Reutilización:** Se pueden reutilizar objetos y definir relaciones entre ellos (herencia).
 - **Mantenibilidad:** Los cambios internos de un objeto bien diseñado tienen menos impacto en otras partes del sistema.
 - **Modelado Natural:** Permite representar entidades del dominio del problema de forma más intuitiva.

(1.2) Los Cuatro Pilares de la POO (Visión General)

La POO se sustenta en cuatro conceptos fundamentales, conocidos como sus pilares:

1. Abstracción:

- *¿Qué es?* Concentrarse en las características *esenciales* de un objeto o entidad, ignorando los detalles irrelevantes o complejos para el contexto actual. Es definir *qué* hace un objeto, sin preocuparse inicialmente por *cómo* lo hace internamente.
- *¿Por qué?* Simplifica el diseño y la comprensión. Permite interactuar con objetos a través de interfaces simples y bien definidas. Ejemplo: Para conducir un auto, necesitas saber usar el

volante, pedales y palanca de cambios (abstracción), no necesitas conocer la ingeniería detallada del motor.

2. Encapsulamiento:

- ¿Qué es? Agrupar los datos (atributos/campos) y los métodos (comportamientos) que operan sobre esos datos dentro de una misma unidad (la *clase*). Además, implica **ocultar** el estado interno del objeto, exponiendo solo lo necesario a través de una interfaz pública (métodos y propiedades).
- ¿Por qué? Protege la integridad de los datos (evita modificaciones inválidas desde fuera), reduce el acoplamiento (cambios internos no afectan tanto al exterior) y facilita la evolución del sistema. Es como una cápsula que protege su contenido.

3. Herencia:

- ¿Qué es? Mecanismo que permite crear nuevas clases (clases *derivadas* o *hijas*) a partir de clases existentes (clases *base* o *padres*). La clase derivada *hereda* los atributos y métodos de la clase base.
- ¿Por qué? Promueve la **reutilización de código**. Permite crear jerarquías de clases ("ES-UN", por ejemplo, un *Perro* ES-UN *Animal*), modelando relaciones de especialización.

4. Polimorfismo:

- ¿Qué es? Literalmente "muchas formas". Es la habilidad de objetos de diferentes clases (relacionadas por herencia o implementación de interfaces) de responder al mismo mensaje (llamada a método) de manera específica a su propio tipo.
- ¿Por qué? Permite escribir código más genérico y flexible. Puedes tratar diferentes objetos de forma uniforme a través de una referencia de tipo base, y el comportamiento correcto se ejecutará automáticamente según el tipo real del objeto.

(1.3) Clases: Las Plantillas para los Objetos

Una *clase* es la definición abstracta, la plantilla o el molde a partir del cual crearemos objetos concretos. Define la estructura (qué datos tendrá) y el comportamiento (qué acciones podrá realizar) de los objetos de ese tipo.

• ¿Cómo se define en C#?

```
// Modificador de acceso (usualmente public) | class | NombreDeLaClase
(PascalCase)
public class Alumno
{
    // Miembros de la clase van aquí dentro {}

    // Campos (Fields) - Almacenan el estado interno (datos)
    // Por convención y encapsulamiento, suelen ser privados
    private string nombre;
    private int legajo;
    private double promedio;

    // Métodos (Methods) - Definen el comportamiento (acciones)
    public void Estudiar(string materia)
    {
        Console.WriteLine($"{nombre} está estudiando {materia}.");
    }
}
```

```

    public void MostrarInformacion()
    {
        Console.WriteLine($"Alumno: {nombre}, Legajo: {legajo}, Promedio: {promedio}");
    }

    // Más adelante veremos Constructores y Propiedades aquí también
}

```

- *Campos (Fields)*: Variables declaradas dentro de la clase para guardar los datos específicos de cada objeto. Se recomienda declararlos como *private* para un buen encapsulamiento.
- *Métodos (Methods)*: Funciones definidas dentro de la clase que operan sobre los datos del objeto o realizan acciones relacionadas con él.

(1.4) Objetos: Las Instancias Concretas

Un *objeto* es una instancia específica y concreta de una clase. Mientras que la clase *Alumno* es la plantilla, cada alumno individual (*alumnoJuan*, *alumnoMaria*) es un objeto creado a partir de esa plantilla. Cada objeto tiene su propia copia de los campos definidos en la clase (su propio *nombre*, *legajo*, *promedio*).

- **¿Cómo se crean (instancian) objetos en C#?** Se utiliza la palabra clave *new*, seguida del nombre de la clase y paréntesis (*new*). Esto realiza dos cosas:
 1. Reserva memoria para almacenar el nuevo objeto.
 2. Llama a un método especial llamado *constructor* (lo veremos en detalle) para inicializar el estado del objeto.

```

// Crear (instanciar) dos objetos de la clase Alumno
Alumno alumnoJuan = new Alumno();
Alumno alumnoMaria = new Alumno();

```

Ahora *alumnoJuan* y *alumnoMaria* son dos objetos distintos, cada uno con su propio espacio en memoria para *nombre*, *legajo* y *promedio*.

- **Acceso a Miembros:** Una vez creado un objeto, accedemos a sus miembros *públicos* (métodos o propiedades) usando el operador punto (*.*).

```

// Asignar valores (aún no podemos directamente a campos privados)
// alumnoJuan.nombre = "Juan Perez"; // ¡ERROR si nombre es privado!

// Llamar a métodos públicos
alumnoJuan.Estudiar("Matemáticas"); // Salida: [Nombre del alumno] está estudiando Matemáticas.
alumnoMaria.Estudiar("Programación");

```

(1.5) Encapsulamiento en Profundidad: Protegiendo el Estado

Como vimos, es buena práctica declarar los campos como *private*. Pero entonces, ¿cómo podemos leer o modificar esos datos desde fuera de la clase cuando sea necesario? La respuesta es mediante **Propiedades**.

- **Modificadores de Acceso:** Controlan la visibilidad de los miembros de una clase.
 - *public*: Accesible desde cualquier lugar (dentro o fuera de la clase).
 - *private*: Accesible *solo* desde dentro de la misma clase. Es el nivel más restrictivo y el recomendado para los campos.
 - *protected*: Accesible desde dentro de la misma clase y desde clases *derivadas* (lo veremos con Herencia).
 - *internal*: Accesible desde cualquier lugar *dentro del mismo ensamblado* (proyecto).
 - *protected internal*: Combinación de *protected* e *internal*.
 - *private protected*: Accesible desde la clase y desde clases derivadas *dentro del mismo ensamblado*.
 - ¿Por qué usar *private* para campos?
 - **Ocultamiento de Información:** El código externo no necesita saber *cómo* se almacenan los datos internamente.
 - **Control:** Permite validar o procesar los datos antes de asignarlos o devolverlos.
 - **Flexibilidad:** Puedes cambiar la implementación interna (ej: el tipo de dato de un campo) sin afectar el código que usa la clase, siempre que la interfaz pública (propiedades, métodos) se mantenga.
- **Propiedades (Properties):** Son miembros de la clase que proporcionan un acceso controlado a los campos privados. Parecen campos desde fuera, pero en realidad son como "métodos disfrazados".
 - **Sintaxis Completa (con campo de respaldo explícito):**

```
public class Alumno
{
    private string nombre; // Campo privado (backing field)
    private int legajo;
    private double promedio;

    // Propiedad pública para el campo 'nombre'
    public string Nombre
    {
        get { return nombre; } // Accesor 'get': devuelve el valor del
        campo
        set { nombre = value; } // Accesor 'set': asigna el valor
        recibido (keyword 'value') al campo
    }

    // Propiedad pública para 'legajo' (solo lectura quizás?)
    public int Legajo
    {
        get { return legajo; }
        // Si no hay 'set', la propiedad es de solo lectura desde fuera
    }

    // Propiedad pública para 'promedio' con validación en set
    public double Promedio
    {
```

```

    get { return promedio; }
    set
    {
        if (value >= 0 && value <= 10) // Validación
        {
            promedio = value;
        }
        else
        {
            // Opción 1: Lanzar un error (excepción)
            // Opción 2: Ignorar el valor inválido
            // Opción 3: Asignar un valor por defecto
            Console.WriteLine("Error: El promedio debe estar entre
0 y 10.");
        }
    }
}
// ... Métodos ...
}

```

- **Propiedades Autoimplementadas (Auto-Properties):** Si no necesitas lógica especial (como validación) en los accesores *get* o *set*, C# ofrece una sintaxis más corta. El compilador genera automáticamente un campo privado oculto (backing field).

```

public class Producto
{
    // Propiedad autoimplementada para Codigo (lectura/escritura)
    public int Codigo { get; set; }

    // Propiedad autoimplementada para Descripcion
    public string Descripcion { get; set; }

    // Propiedad autoimplementada para Precio, pero con 'set' privado
    // Solo se puede asignar el precio desde dentro de la clase
    Producto
    public double Precio { get; private set; }

    // ... Constructores y métodos podrían asignar Precio ...
    public Producto(int codigo, string desc, double precioInicial)
    {
        Codigo = codigo;
        Descripcion = desc;
        Precio = precioInicial; // Válido porque estamos dentro de la
clase
    }

    public void AplicarDescuento(double porcentaje)
    {
        if(porcentaje > 0 && porcentaje < 100)
        {
            Precio = Precio * (1 - porcentaje / 100.0); // Válido
        }
    }
}

```

```
    }
}
```

- ¿Cómo usar propiedades desde fuera? Exactamente igual que si fueran campos públicos:

```
Alumno alu = new Alumno();
alu.Nombre = "Maria Garcia"; // Usa el accesor 'set' de la propiedad
Nombre
string nombreAlumno = alu.Nombre; // Usa el accesor 'get' de la
propiedad Nombre

// alu.Legajo = 1234; // ¡ERROR si Legajo no tiene 'set' público!
// int legajoAlumno = alu.Legajo; // OK, usa el 'get' público

alu.Promedio = 11; // Imprimirá el mensaje de error desde el 'set'
alu.Promedio = 8.5; // OK, asigna el valor al campo privado 'promedio'
```

Parte 2: Construyendo y Organizando Objetos

(2.1) Constructores: Dando Vida a los Objetos

Cuando usamos *new* para crear un objeto, se llama automáticamente a un método especial de la clase: el **constructor**.

- **¿Cuál es su propósito principal?** Inicializar el estado del objeto recién creado, asegurando que comience con valores válidos y consistentes. Es el lugar ideal para asignar valores iniciales a los campos privados.
- **Características:**
 - Tiene exactamente el **mismo nombre** que la clase.
 - **No tiene tipo de retorno**, ni siquiera *void*.
- **Constructor por Defecto:** Si *no* escribes ningún constructor en tu clase, C# genera uno automáticamente. Este constructor es *public* y no tiene parámetros ni cuerpo (no hace nada explícitamente, pero permite crear el objeto y los campos toman sus valores por defecto: 0 para números, *false* para *bool*, *null* para referencias).
 - **¡Importante!** En el momento en que defines *cualquier* constructor (con o sin parámetros), el constructor por defecto **ya no se genera automáticamente**. Si todavía quieres poder crear objetos sin pasar argumentos, tendrás que definir explícitamente un constructor sin parámetros:


```
public NombreClase() { /* inicialización si se necesita */ }
```
- **Constructores con Parámetros:** Permiten pasar valores al crear el objeto para inicializar sus campos.

```
public class Alumno
{
    // Campos privados (igual que antes)
    private string nombre;
    private int legajo;
    // ...
}
```

```
// Constructor SIN parámetros (explícito)
public Alumno()
{
    nombre = "NN"; // Asignar valor por defecto
    legajo = 0;    // Asignar valor por defecto
    Console.WriteLine("Objeto Alumno creado con constructor por
defecto.");
}

// Constructor CON parámetros
public Alumno(string nombreInicial, int legajoInicial)
{
    nombre = nombreInicial; // Asignar desde parámetro
    legajo = legajoInicial; // Asignar desde parámetro
    Console.WriteLine($"Objeto Alumno creado para {nombre}.");
}

// ... Propiedades y Métodos ...
}
```

- Uso:

```
Alumno a1 = new Alumno(); // Llama al constructor sin parámetros
Alumno a2 = new Alumno("Pedro Lopez", 5678); // Llama al constructor
con parámetros
```

- **Sobrecarga de Constructores (Overloading):** Puedes tener múltiples constructores en la misma clase, siempre que se diferencien en la **cantidad** o el **tipo** de sus parámetros. C# sabrá cuál llamar basándose en los argumentos que pases al usar *new*.
- **La Palabra Clave *this*:**
 1. **Desambiguación:** Si un parámetro de un constructor (o método) tiene el mismo nombre que un campo de la clase, puedes usar *this.nombreCampo* para referirte explícitamente al campo de la instancia actual.

```
public Alumno(string nombre, int legajo)
{
    // 'this.nombre' es el campo de la clase, 'nombre' es el parámetro
    this.nombre = nombre;
    this.legajo = legajo;
}
```

2. **Llamada a otro Constructor (Constructor Chaining):** Un constructor puede llamar a otro constructor *de la misma clase* usando *this(...)* después de su propia lista de parámetros. Esto es útil para evitar repetir código de inicialización. La llamada a *this(...)* debe ser lo primero que se ejecuta.

```

public class Alumno
{
    // ... campos ...

    // Constructor principal que hace la inicialización real
    public Alumno(string nombre, int legajo)
    {
        this.nombre = nombre;
        this.legajo = legajo;
        Console.WriteLine("Constructor principal ejecutado.");
    }

    // Constructor sin parámetros que llama al principal con valores
    // por defecto
    public Alumno() : this("Sin Nombre", 0) // Llama al constructor de
    arriba
    {
        Console.WriteLine("Constructor por defecto (que llama al otro)
        ejecutado.");
        // Ya no necesito repetir: this.nombre = "Sin Nombre";
        this.legajo = 0;
    }

    // ... Propiedades y Métodos ...
}

```

(2.2) Miembros Estáticos: Compartidos por Todos

Normalmente, los campos y métodos que definimos pertenecen a *cada instancia* (objeto) de la clase. Pero a veces necesitamos miembros que pertenezcan a la *clase en sí misma*, compartidos por todos los objetos de esa clase, o accesibles incluso sin haber creado ningún objeto. Estos son los miembros **estáticos**, marcados con la palabra clave *static*.

- **Campos Estáticos (static fields):**

- Existe *una sola copia* de este campo en memoria para toda la clase, compartida por todas las instancias.
- ¿Cuándo usarlos?
 - Contadores globales para la clase (ej: cuántos objetos *Alumno* se han creado).
 - Constantes compartidas (aunque *const* es para valores fijos en tiempo de compilación y *static readonly* es para valores fijos en tiempo de ejecución).
 - Puntos de acceso a instancias únicas (patrón Singleton, más avanzado).

```

public class Alumno
{
    // ... campos de instancia (nombre, legajo) ...

    // Campo ESTÁTICO para contar instancias
    private static int contadorAlumnos = 0;
}

```



```

public Alumno(string nombre, int legajo)
{
    this.nombre = nombre;
    this.legajo = legajo;
    contadorAlumnos++; // Incrementar el contador estático CADA VEZ que
se crea un objeto
    Console.WriteLine($"Se creó el alumno #{contadorAlumnos}");
}
// ...
}

```

- **Métodos Estáticos (static methods):**

- Se llaman directamente usando el nombre de la clase, sin necesidad de crear un objeto. Ejemplo: `Console.WriteLine()`, `Math.Max(a, b)`.
- **Importante:** Un método estático *no* opera sobre una instancia específica, por lo tanto, *no* puede acceder directamente a campos o métodos *de instancia* (no estáticos) de la clase. Sí puede acceder a otros miembros *estáticos*.
- ¿Cuándo usarlos?
 - Métodos de utilidad que no dependen del estado de un objeto particular (ej: métodos matemáticos, de conversión, de validación genéricos).
 - Para acceder o modificar campos estáticos privados (ej: un método estático para obtener el contador de alumnos).
 - Métodos "fábrica" que crean y devuelven instancias de la clase.

```

public class Alumno
{
    // ... campos (incluido 'contadorAlumnos' estático) y constructores ...

    // Método ESTÁTICO para obtener el contador
    public static int ObtenerCantidadAlumnos()
    {
        return contadorAlumnos; // Accede al campo estático
        // NO PUEDE hacer esto: return this.nombre; (porque no hay 'this')
    }

    // Método de instancia (NO estático)
    public void MostrarInformacion()
    {
        // Puede acceder a miembros de instancia Y estáticos
        Console.WriteLine($"Info: {nombre}, Legajo: {legajo}. Total alumnos:
{contadorAlumnos}");
    }
    // ...
}

// Uso desde Main (que es estático):
// ... crear alumnos a1, a2 ...
int total = Alumno.ObtenerCantidadAlumnos(); // Llamada al método estático
Console.WriteLine($"Total de alumnos creados: {total}");
a1.MostrarInformacion(); // Llamada a método de instancia

```

(2.3) Organizando Objetos: Introducción a Colecciones (`List<T>`)

Ya sabemos crear objetos, pero ¿cómo manejamos varios objetos juntos? Podríamos usar *arrays*, pero tienen una limitación grande: su **tamaño es fijo**. Si creamos un `Alumno[]` de tamaño 10, no podemos agregar fácilmente al alumno número 11.

Aquí entran las **Colecciones Genéricas** del *namespace* `System.Collections.Generic`. La más común y útil para empezar es `List<T>`.

- ¿Qué es `List<T>`?

- Es una clase que representa una **lista ordenada de elementos** cuyo **tamaño puede crecer o disminuir dinámicamente** según necesites.
- Es **genérica**, indicada por la `<T>`. La `T` es un marcador de posición para el *tipo* de elementos que almacenará la lista. Esto proporciona **seguridad de tipos**: una `List<Alumno>` solo puede contener objetos `Alumno` (o de clases derivadas), no puedes agregarle un `Producto` por error. El compilador lo verifica.

- Operaciones Básicas:

```
// Incluir el namespace al principio del archivo:
using System.Collections.Generic;

// ... dentro de Main u otro método ...

// 1. Crear una lista VACÍA de Alumnos
List<Alumno> listaDeAlumnos = new List<Alumno>();

// 2. Crear algunos objetos Alumno
Alumno a1 = new Alumno("Ana Sol", 111);
Alumno a2 = new Alumno("Luis Mar", 222);
Alumno a3 = new Alumno("Eva Luna", 333);

// 3. Añadir objetos a la lista
listaDeAlumnos.Add(a1);
listaDeAlumnos.Add(a2);
listaDeAlumnos.Add(a3);
// Ahora la lista contiene 3 alumnos

// 4. Obtener la cantidad de elementos
Console.WriteLine($"Cantidad de alumnos en la lista:
{listaDeAlumnos.Count}"); // Imprime 3

// 5. Acceder a un elemento por su índice (base 0)
Alumno primerAlumno = listaDeAlumnos[0]; // Obtiene a Ana Sol
Console.WriteLine($"El primer alumno es: {primerAlumno.Nombre}"); //
Asumiendo que Nombre es una propiedad pública

// 6. Recorrer la lista con foreach (la forma más común)
Console.WriteLine("\nLista Completa:");
foreach (Alumno alu in listaDeAlumnos)
{
```

```
// 'alu' toma el valor de cada elemento de la lista en cada iteración
alu.MostrarInformacion(); // Llama al método del objeto Alumno actual
}

// 7. (Otras operaciones útiles: Remove, Insert, Contains, Clear, etc.)
// listaDeAlumnos.Remove(a2); // Eliminar a Luis Mar
// bool existeEva = listaDeAlumnos.Contains(a3); // Verificar si Eva está
```

- ¿Por qué `List<T>` es tan usada? Porque combina la flexibilidad de tamaño con la seguridad de tipos que ofrecen los genéricos, haciendo mucho más fácil manejar colecciones de objetos.

Parte 3: Construyendo Jerarquías - Herencia

La herencia es uno de los pilares más potentes de la POO. Nos permite modelar relaciones de tipo "ES-UN" y reutilizar código de forma efectiva.

(3.1) El Principio de Herencia: Reutilización y Especialización

- **Concepto:** Permite definir una clase nueva (*derivada*) que hereda características (campos y métodos) de una clase existente (*base*). La clase derivada puede luego añadir sus propios miembros específicos o modificar (sobrescribir) el comportamiento heredado.
- **Relación "ES-UN" (IS-A):** Es la clave para decidir si la herencia es apropiada. Si puedes decir "un objeto de la ClaseB ES UN objeto de la ClaseA", entonces B probablemente debería heredar de A.
 - Ejemplos: Un *Alumno* ES UNA *Persona*. Un *Profesor* ES UNA *Persona*. Un *Auto* ES UN *Vehiculo*. Un *Perro* ES UN *Mamifero* que ES UN *Animal*.
- **Beneficios:**
 - **Reutilización de Código:** Evita escribir los mismos campos y métodos comunes en múltiples clases. Se definen una vez en la clase base.
 - **Especialización:** Permite crear versiones más específicas de una clase general.
 - **Creación de Jerarquías:** Organiza las clases en una estructura de árbol que refleja relaciones del mundo real o conceptual.
 - **Polimorfismo:** La herencia es la base para que funcione el polimorfismo (lo veremos en la Parte 4).

(3.2) Implementando Herencia en C#

- **Sintaxis:** Se usa el símbolo dos puntos (`:`) después del nombre de la clase derivada, seguido del nombre de la clase base.

```
// Clase Base (Padre / Superclase)
public class Persona
{
    public string Nombre { get; set; }
    public string Apellido { get; set; }
    protected string DNI { get; set; } // Protected: accesible aquí y en
    derivadas

    public void Caminar()
    {
```

```

        Console.WriteLine($"{Nombre} está caminando.");
    }

    // Método que PODRÁ ser sobrescrito por las derivadas
    public virtual void Presentarse()
    {
        Console.WriteLine($"Hola, soy {Nombre} {Apellido}.");
    }
}

// Clase Derivada (Hija / Subclase)
public class Alumno : Persona // Alumno HEREDA de Persona
{
    public int Legajo { get; set; }

    // Método propio de Alumno
    public void Estudiar()
    {
        // Puede acceder a miembros public y protected de Persona
        Console.WriteLine($"El alumno {Nombre} (Legajo: {Legajo}, DNI:
{DNI}) está estudiando.");
        // Console.WriteLine(this.DNI); // También funciona
    }

    // Sobrescribiendo el método Presentarse de la base
    public override void Presentarse()
    {
        // Puedo llamar a la implementación base si quiero:
        base.Presentarse();
        Console.WriteLine($"Soy el alumno {Nombre} {Apellido}, legajo
{Legajo}.");
    }
}

// Otra Clase Derivada
public class Profesor : Persona
{
    public string Materia { get; set; }

    public void Enseñar()
    {
        Console.WriteLine($"El profesor {Nombre} está enseñando
{Materia}.");
    }

    public override void Presentarse()
    {
        Console.WriteLine($"Soy el profesor {Nombre} {Apellido}, dicto
{Materia}.");
    }
}

```

- ¿Qué se Hereda?

- Todos los miembros *public* y *protected* de la clase base (campos, propiedades, métodos).
- Los miembros *private* de la base existen en el objeto derivado, pero **no son directamente accesibles** desde el código de la clase derivada.
- **Modificador *protected***: Permite el acceso desde la propia clase y desde cualquier clase que herede de ella, pero no desde fuera. Es útil para miembros que son detalles internos de la jerarquía pero no deben ser públicos.
- **Herencia Simple**: C# solo permite heredar de *una única clase base* directamente. Esto evita problemas de ambigüedad presentes en lenguajes con herencia múltiple (como C++). Se pueden crear cadenas de herencia (A hereda de *Object*, B hereda de A, C hereda de B).

(3.3) Constructores en la Herencia: La Llamada a *base*

- Los constructores **no se heredan**. Cada clase es responsable de su propia inicialización.
- Sin embargo, antes de que se ejecute el cuerpo del constructor de una clase derivada, **se debe ejecutar un constructor de su clase base**. ¿Por qué? Para asegurar que la parte "base" del objeto esté correctamente inicializada antes de inicializar la parte "derivada".
- **Llamada Implícita**: Si el constructor de la derivada *no* llama explícitamente a un constructor base usando `: base(...)`, C# intenta llamar automáticamente al constructor *sin parámetros* de la clase base. Si la clase base no tiene un constructor sin parámetros accesible, ¡tendrás un error de compilación!
- **Llamada Explícita con `: base(...)`**: Para llamar a un constructor específico de la clase base (generalmente uno con parámetros), usas la sintaxis `: base(argumentos)` justo después de la firma del constructor de la derivada y antes de su bloque `{}`.

```
public class Persona
{
    public string Nombre { get; set; }
    public string Apellido { get; set; }

    // Constructor de Persona
    public Persona(string nombre, string apellido)
    {
        Console.WriteLine("Ejecutando constructor de Persona");
        Nombre = nombre;
        Apellido = apellido;
    }
}

public class Alumno : Persona
{
    public int Legajo { get; set; }

    // Constructor de Alumno LLAMANDO al constructor de Persona
    public Alumno(string nombre, string apellido, int legajo)
        : base(nombre, apellido) // Pasa nombre y apellido al constructor
base
    {
        // El constructor base se ejecuta PRIMERO
        Console.WriteLine("Ejecutando constructor de Alumno");
        // Ahora inicializa los miembros propios de Alumno
        Legajo = legajo;
    }
}
```

```

    }
}

// Uso:
// Alumno a = new Alumno("Carlos", "Gomez", 999);
// Salida:
// Ejecutando constructor de Persona
// Ejecutando constructor de Alumno

```

(3.4) Sobrescritura de Métodos: **virtual** y **override**

La herencia nos permite reutilizar, pero a veces necesitamos que la clase derivada haga algo *diferente* a lo que hace la clase base para un mismo método.

- **¿Cómo permitir la sobrescritura?** La clase **base** debe marcar el método como *virtual*. Esto indica que las clases derivadas *tienen permiso* para proporcionar su propia implementación.
- **¿Cómo sobrescribir?** La clase **derivada** debe marcar el método con la palabra clave *override* y proporcionar la nueva implementación. El método sobrescrito debe tener exactamente la misma firma (nombre, parámetros, tipo de retorno) que el método *virtual* de la base.

```

public class Animal
{
    public string Nombre { get; set; }

    // Método virtual: las clases derivadas PUEDEN sobrescribirlo
    public virtual void HacerSonido()
    {
        Console.WriteLine("El animal hace un sonido genérico.");
    }
}

public class Perro : Animal
{
    // Sobrescribe el método de la base
    public override void HacerSonido()
    {
        Console.WriteLine("El perro hace: ¡Guau! ¡Guau!");
    }
}

public class Gato : Animal
{
    // Sobrescribe con otra implementación
    public override void HacerSonido()
    {
        Console.WriteLine("El gato hace: ¡Miau!");
    }
}

// Uso:
// Perro p = new Perro();

```

```
// Gato g = new Gato();
// p.HacerSonido(); // Salida: El perro hace: ¡Guau! ¡Guau!
// g.HacerSonido(); // Salida: El gato hace: ¡Miau!
```

- **Llamar a la Implementación Base:** Dentro de un método *override*, a veces quieres ejecutar la lógica original de la clase base *además* de la tuya. Puedes hacerlo usando `base.NombreMetodo()`.

```
public class Profesor : Persona
{
    // ...
    public override void Presentarse()
    {
        base.Presentarse(); // Llama al Presentarse() de Persona
        Console.WriteLine($"Y dicto la materia {Materia}."); // Añade info
        // específica
    }
}
```

Parte 4: Conceptos Avanzados - Polimorfismo, Abstracción y Contratos

(4.1) Polimorfismo en Acción: El Poder de las "Muchas Formas"

El polimorfismo es donde la herencia y la sobrescritura realmente brillan. Nos permite escribir código que opera sobre objetos de la clase base, pero que se comporta correctamente según el tipo *real* (derivado) del objeto en tiempo de ejecución.

- **La Clave:** Una variable declarada del tipo de la clase **base** puede almacenar una referencia a un objeto de **cualquiera de sus clases derivadas**.

```
// 'p1' es de tipo Persona, pero apunta a un objeto Alumno
Persona p1 = new Alumno("Laura", "Paz", 123);

// 'p2' es de tipo Persona, pero apunta a un objeto Profesor
Persona p2 = new Profesor("Roberto", "Diaz", "Historia");

// Incluso podemos tener una lista de Personas con diferentes tipos reales
List<Persona> plantel = new List<Persona>();
plantel.Add(p1);
plantel.Add(p2);
plantel.Add(new Alumno("Sergio", "Cruz", 456));
```

- **La Magia (Dynamic Dispatch / Late Binding):** Cuando llamas a un método *virtual* (que ha sido sobrescrito - *override*) a través de una referencia de tipo base, .NET no ejecuta el método de la clase base (*Persona*). En cambio, en **tiempo de ejecución**, mira el tipo *real* del objeto al que apunta la referencia y ejecuta la versión *override* correspondiente a esa clase derivada.

```

Console.WriteLine("Presentaciones:");
foreach (Persona p in plantel)
{
    // Aunque 'p' es de tipo Persona, aquí se llamará al
    // Presentarse() de Alumno o al de Profesor, según el objeto real!
    p.Presentarse();
}

```

Salida Esperada:

```

Presentaciones:
Soy el alumno Laura Paz, legajo 123.
Soy el profesor Roberto Diaz, dicto Historia.
Soy el alumno Sergio Cruz, legajo 456.

```

- **¿Por qué es tan poderoso el Polimorfismo?**

- **Flexibilidad y Extensibilidad:** Puedes añadir nuevas clases derivadas (ej: *Director*, *Preceptor* que hereden de *Persona*) sin tener que modificar el código que trabaja con la lista de *Persona*. El bucle *foreach* seguirá funcionando y llamando al método *Presentarse()* correcto para los nuevos tipos.
- **Código Genérico:** Permite escribir métodos que operen sobre la abstracción (la clase base) y funcionen correctamente con cualquier implementación concreta (las clases derivadas).

(4.2) Clases Abstractas: Definiendo Conceptos Incompletos

A veces, una clase base representa un concepto tan general que no tiene sentido crear instancias directas de ella (ej: ¿qué es un "Animal" genérico sin ser un perro, gato, etc.? ¿Qué es una "FiguraGeometrica" sin ser un círculo o cuadrado?). Además, puede que queramos *obligar* a todas las clases derivadas a implementar ciertos métodos, pero no podemos proporcionar una implementación por defecto en la base. Para esto existen las **clases abstractas**.

- **Características:**

- Se marcan con la palabra clave *abstract*.
- **No se pueden instanciar** directamente con *new*.
- Pueden contener miembros normales (campos, métodos implementados, métodos virtuales).
- Pueden contener **métodos abstractos**.

- **Métodos Abstractos:**

- Se marcan con *abstract*.
- **No tienen cuerpo** (sin `{}`), solo la firma terminada en `;`.
- **Deben ser implementados (sobrescritos con *override*)** por cualquier clase derivada *concreta* (no abstracta) que herede de la clase abstracta.
- Son implícitamente *virtuales*.

```

// Clase abstracta: no se puede hacer 'new FiguraGeometrica()'
public abstract class FiguraGeometrica
{

```



```

    public string Color { get; set; }

    // Método abstracto: SIN implementación. Obliga a las derivadas a
    definirlo.
    public abstract double CalcularArea();

    // Método normal (no abstracto)
    public void MostrarColor()
    {
        Console.WriteLine($"Color: {Color}");
    }
}

public class Circulo : FiguraGeometrica
{
    public double Radio { get; set; }

    // Obligado a implementar CalcularArea
    public override double CalcularArea()
    {
        return Math.PI * Radio * Radio;
    }
}

public class Rectangulo : FiguraGeometrica
{
    public double Base { get; set; }
    public double Altura { get; set; }

    // Obligado a implementar CalcularArea
    public override double CalcularArea()
    {
        return Base * Altura;
    }
}

```

- ¿Cuándo usar clases abstractas? Cuando tienes una jerarquía "ES-UN" clara, quieres compartir código común en la base, pero la base es demasiado abstracta para existir por sí sola y/o necesitas forzar a las derivadas a implementar ciertos comportamientos esenciales.

(4.3) Interfaces: Definiendo Contratos de Capacidades

Mientras que la herencia de clases define una relación "ES-UN", las **interfaces** definen un **contrato** de lo que una clase *puede hacer* (una capacidad, un rol), sin importar su posición en la jerarquía de herencia.

- **Características:**
 - Se definen con la palabra clave *interface*. Por convención, sus nombres empiezan con **I** (ej: *IDisposable*, *IEnumerable*, *IMiInterfaz*).
 - Contienen **firmas** de métodos, propiedades, eventos o indexadores. **Tradicionalmente no contienen implementación** (aunque versiones recientes de C# permiten implementaciones por defecto, es mejor empezar sin ellas).

- No se pueden instanciar directamente (`new IInterfaz()` es ilegal).
- Una clase (o struct) puede **implementar** una o **múltiples** interfaces.
- Si una clase implementa una interfaz, **debe proporcionar una implementación pública** para *todos* los miembros definidos en esa interfaz.

- **Sintaxis:**

```
// Definición de la interfaz
public interface IDibujable
{
    // Contrato: cualquier clase que implemente IDibujable DEBE tener este
    método
    void Dibujar();

    // También puede definir firmas de propiedades
    int PosicionX { get; set; }
    int PosicionY { get; set; }
}

// Clase que implementa la interfaz
public class Boton : IDibujable // Boton implementa IDibujable
{
    // Debe implementar TODOS los miembros de IDibujable
    public int PosicionX { get; set; }
    public int PosicionY { get; set; }

    public void Dibujar()
    {
        Console.WriteLine($"Dibujando Botón en ({PosicionX}, {PosicionY})");
    }

    // ... otros miembros propios de Boton ...
}

// Otra clase, quizás no relacionada con Boton, también puede ser dibujable
public class Imagen : IDibujable
{
    public int PosicionX { get; set; }
    public int PosicionY { get; set; }
    public string RutaArchivo { get; set; }

    public void Dibujar()
    {
        Console.WriteLine($"Dibujando Imagen '{RutaArchivo}' en
        ({PosicionX}, {PosicionY})");
    }
}
```

- **Polimorfismo con Interfaces:** Similar a las clases base, puedes tener referencias o colecciones del tipo de la interfaz que contengan objetos de cualquier clase que implemente esa interfaz.

```

List<IDibujable> elementosEnPantalla = new List<IDibujable>();
elementosEnPantalla.Add(new Boton { PosicionX = 10, PosicionY = 20 });
elementosEnPantalla.Add(new Imagen { PosicionX = 50, PosicionY = 60,
RutaArchivo = "logo.png" });

foreach(IDibujable elem in elementosEnPantalla)
{
    elem.Dibujar(); // Llama al método Dibujar() específico de Boton o
Imagen
}

```

- **¿Cuándo usar Interfaces vs. Clases Abstractas?**

- Usa **Herencia (con Clases Abstractas si es necesario)** cuando hay una relación clara de tipo **"ES-UN"** y quieres **reutilizar código** (implementaciones) de la clase base.
- Usa **Interfaces** cuando quieres definir una **capacidad** o **rol ("PUEDE-HACER")** que puede ser compartido por clases no necesariamente relacionadas jerárquicamente, o cuando una clase necesita adoptar **múltiples contratos**. Las interfaces promueven un diseño más desacoplado.

(4.4) **System.Object**: El Ancestro Universal

En .NET, todas las clases (y de hecho, todos los tipos, incluyendo *structs* y tipos primitivos mediante *boxing*) heredan implícitamente de una clase base fundamental: **System.Object**.

- **¿Qué significa esto?** Cualquier objeto en C#, no importa de qué clase sea, "ES UN" *Object*. Esto permite, por ejemplo, crear colecciones genéricas que puedan contener cualquier cosa (aunque es mejor usar colecciones tipadas como *List<T>* por seguridad).
- **Métodos Heredados Comunes:** *Object* proporciona algunos métodos básicos que están disponibles en todos los objetos:
 - ***ToString()***: Devuelve una representación en forma de *string* del objeto. Por defecto, devuelve el nombre completo del tipo de la clase. Es *virtual*, por lo que **es muy común sobrescribirlo (*override*)** en tus propias clases para proporcionar una representación más útil y legible.
 - ***Equals(object obj)***: Compara el objeto actual con otro objeto para ver si son iguales. La implementación por defecto para tipos de referencia compara si apuntan a la misma instancia en memoria. Se puede sobrescribir para definir igualdad basada en el estado (valor) de los objetos.
 - ***GetHashCode()***: Devuelve un número entero (código hash) que representa al objeto. Se usa internamente por colecciones como *Dictionary* o *HashSet* para buscar elementos eficientemente. Si sobrescribes *Equals*, generalmente también debes sobrescribir *GetHashCode* para mantener la consistencia.
 - ***GetType()***: Devuelve un objeto *Type* que contiene información sobre el tipo real del objeto en tiempo de ejecución (reflection).
- **Sobrescribiendo *ToString()* (¡Muy útil!)**

```

public class Alumno // : Persona (hereda de Persona, que hereda de Object)
{
    // ... campos, propiedades, constructores, otros métodos ...

    // Sobrescribiendo ToString() para una mejor representación

```

```

public override string ToString()
{
    // Devuelve un string formateado con la info del alumno
    return $"Legajo: {Legajo} - Nombre: {Nombre} {Apellido}";
}

// Uso:
// Alumno a1 = new Alumno("Ana", "Sol", 111);
// Console.WriteLine(a1); // Llama implícitamente a a1.ToString()
// Salida: Legajo: 111 - Nombre: Ana Sol

```

Sobrescribir *ToString()* es excelente para depuración y para mostrar información de objetos de forma rápida y legible.

Conclusión

La Programación Orientada a Objetos es un paradigma poderoso que, aunque requiere un cambio en la forma de pensar respecto a la programación procedural, ofrece enormes beneficios en términos de organización, reutilización, mantenibilidad y flexibilidad para construir software complejo y robusto. Dominar los conceptos de Clases, Objetos, Encapsulamiento, Herencia, Polimorfismo, Clases Abstractas e Interfaces es esencial para cualquier desarrollador .NET moderno. La práctica constante en el modelado y la implementación de estos conceptos solidificará tu comprensión.

Síntesis de una oración: La Programación Orientada a Objetos en C# organiza el código en clases y objetos que encapsulan datos y comportamiento, utilizando herencia para reutilizar código, polimorfismo para flexibilidad, y abstracción (clases abstractas/interfaces) para definir estructuras y contratos.

Síntesis de 5 puntos importantes:

1. La POO modela el software usando **Clases** (plantillas) y **Objetos** (instancias) que combinan datos y comportamiento, aplicando **Encapsulamiento** (ocultamiento de datos con *private* y acceso controlado con Propiedades).
2. La **Herencia** (: *BaseClass*, *virtual*, *override*) permite crear jerarquías "ES-UN", reutilizar código y especializar clases.
3. El **Polimorfismo** permite tratar objetos derivados a través de referencias de clase base, ejecutando el comportamiento específico de la subclase (late binding), lo que da gran flexibilidad.
4. Las **Clases Abstractas** (*abstract*) definen conceptos base no instanciables y pueden forzar la implementación de métodos en las derivadas.
5. Las **Interfaces** (*interface*) definen contratos de capacidades ("PUEDE-HACER") que las clases deben implementar, permitiendo polimorfismo entre clases no relacionadas y promoviendo bajo acoplamiento.